TASK – 8

Smart Traffic Navigation System Using Java

PROGRAM :

```
import java.util.*;

public class SmartTrafficNavigation {

  static Map<String, Map<String, Integer>> graph =
      new HashMap<>();

  static void addRoad(String a, String b, int time) {
    graph.computeIfAbsent(a, k -> new HashMap<>()).put(b, time);
    graph.computeIfAbsent(b, k -> new HashMap<>()).put(a, time);
  }

  static void findRoute(String source, String destination) {
    Map<String, Integer> distance = new HashMap<>();
    Map<String, String> parent = new HashMap<>();

    PriorityQueue<String> queue = new PriorityQueue<>(
      Comparator.comparingInt(n -> distance.getOrDefault(n, Integer.MAX_VALUE))
    );

    for (String node : graph.keySet()) {
      distance.put(node, Integer.MAX_VALUE);
    }
```

```
distance.put(source, 0);
queue.add(source);

while (!queue.isEmpty()) {
  String current = queue.poll();

  if (current.equals(destination)) break;

  for (Map.Entry<String, Integer> road :
      graph.getOrDefault(current, Collections.emptyMap()).entrySet()) {

    String next = road.getKey();
    int newTime = distance.get(current) + road.getValue();

    if (newTime < distance.get(next)) {
      distance.put(next, newTime);
      parent.put(next, current);
      queue.remove(next);
      queue.add(next);
    }
  }
}

if (distance.getOrDefault(destination, Integer.MAX_VALUE)
    == Integer.MAX_VALUE) {
  System.out.println("No route available.");
  return;
```

```
    }

    List<String> path = new ArrayList<>();
    String current = destination;

    while (current != null) {
        path.add(current);
        current = parent.get(current);
    }

    Collections.reverse(path);

    System.out.println("Best Route: " + String.join(" -> ", path));
    System.out.println("Estimated Travel Time: "
            + distance.get(destination) + " minutes");
}

public static void main(String[] args) {
    addRoad("A", "B", 10);
    addRoad("A", "C", 15);
    addRoad("B", "D", 12);
    addRoad("C", "D", 5);
    addRoad("D", "E", 8);
    addRoad("B", "E", 25);

    Scanner sc = new Scanner(System.in);

    System.out.println("Available Locations: A, B, C, D, E");
```

```
        System.out.print("Enter source: ");
        String source = sc.next().toUpperCase();

        System.out.print("Enter destination: ");
        String destination = sc.next().toUpperCase();

        if (!graph.containsKey(source) ||
              !graph.containsKey(destination)) {
           System.out.println("Invalid location.");
        } else {
           findRoute(source, destination);
        }

        sc.close();
    }
}
```

INPUT :

Enter source: A

Enter destination: E

OUTPUT :

Available Locations: A, B, C, D, E

Best Route: A -> C -> D -> E

Estimated Travel Time: 28 minutes